

Skriftlig uppgift 1

Författare: Lucas Berg

Denna uppgift skall svaras på individuellt, se till att ha en egen design på första uppgiften och egna svar på andra delen. Det är både tillåtet och uppmuntrat att diskutera med klasskamrater och dela källor med varandra!

Skriv in dina svar i detta dokument på respektive platser.

Håll dina svar kortfattade och kärnfulla. Svara gärna i punktlistor. Absolut max 2000 ord total för svaren, men håll det gärna kortare (så om du räknar totala antalet ord i dokument ska det vara under ca 2500).

*Du får godkänt för att svara på frågorna, **inte skriva mycket text!***

Programmeringsparadigm

Du ska designa ett inventory-system i ett spel med följande funktionalitet:

- *I spelet finns ett koncept av en inventory som håller koll på objekt (t ex en bok, en stav, en hatt, en hammare) och resurser (t ex läder, koppar). Varje inventory har ett visst antal platser/slots. I varje slot ligger nåt av: ingenting, ett objekt, 1-10 resurser.*
- *En inventory kan finnas på den egna spelare, kistor i världen och besegrade fiender.*
- *På den egna spelaren finns speciella slots för de saker du aktivt använder: huvudbonad, tröja, byxor, skor, handskar, samt 3 vapen/verktyg.*
- *I spelet kan man med en knapp öppna inventory-uit. Då får man upp sin egen inventory och om man står brevid och vänd mot något i världen med en inventory dess inventory. När det är öppet är spelet pausat.*
- *I inventory-uit kan man:*
 - *Flytta saker mellan slots inom en inventory*
 - *Flytta saker mellan slots i olika inventories (om du ser två)*
 - *Flytta saker till de speciella slotsen för vad man aktivt använder*

- *Dela och slå ihop resurs-högar. T ex du har 10 av en resurs, du kan dela det till två slottar av 5*
 - *Ångra/göra om ändringar (undo/redo) medan du har UIt uppe*
 - *Stänga UIt, då sparas dina ändringar*
- *Kod utanför inventory-systemet ska lätt kunna fråga vad som är aktivt i varje slot.*

Analysera programmet från ett procedurellt, dataorienterat, funktionellt och objektorienterat perspektiv. Vad lär du dig från varje perspektiv?

Procedurellt - att analysera systemet procedurellt ger mig ett väldigt tydligt flöde när det kommer till vad som ska hända och när det ska hända. Att programmera procedurellt innebär att implementera beteenden i programmet via funktioner som kallar på varandra. I slutändan får man ett program som är en serie av steg som formar en hierarki av calls till funktioner (wikipedia, procedural programming, 2026).

Dataorienterat - att analysera systemet dataorienterat visar hur informationen om inventory faktiskt representeras och används i minnet. Att programmera dataorienterat innebär att utgå från strukturerad data (till exempel arrays av slots med object-id och size) och sedan designa funktioner som arbetar effektivt med dessa strukturer. Tillslut får man ett program där layouten av data, cachevänligheten styr hur jag programmerar och organiserar systemet, snarare än objekt (OOP) eller anropshierarkier (procedural) (wikipedia data-oriented design, 2026).

Funktionellt - Genom att analysera systemet funktionellt får jag en bättre förståelse av hur inventorys tillstånd förändras genom transformationer av data. Att programmera funktionellt innebär att använda sig av funktioner som tar nuvarande state och returnerar ett nytt state utan sidoeffekter. Man får då ett program som är en serie av deterministiska transformationer, där undo/redo och testning förenklas eftersom varje steg bara beror på sina indata (wikipedia functional programming, 2026)

Objektorienterat - Att analysera systemet ur ett objektorienterat perspektiv ger mig en tydlig fokus på vad för typ av objekt som behövs samt dess ansvar. Varje objekt kan kapsla in sin data och funktioner, som gör att man undviker sprida logik för en klass till andra klasser. Med arv och polymorfism kan exempelvis Resource och Item dela en gemensam basklass men ändå särskiljas i beteende, som att resources kan stackas medan objekt inte kan det. Det ger även ett tydligt gränssnitt så att resten av spelet enkelt kan fråga de olika objekten, utan att veta om det exempelvis är ett item eller en resource.

Skissa sedan en översikt av ett komplett system. Vad tänker du dig att du behöver för funktioner och data? Hur vill du gruppera ihop det till klasser och structar? OBS Implementera inte funktioner, skissa endast på uppdelning av data och kod.

På detta sättet sorterade jag data och klasserna runt tydliga ansvarsområden: objekten såsom Item och resource, Inventory med Slots, Player och PlayerInventory som bara pratar via gränssnittet såsom GetObjectAt, GetInventory, MoveIntoOtherInventory.

```
using InputManager = CommonUtilities::InputHandler;

class GameWorld;

class ObjectBase
{
public:
    ObjectBase();

    void Update(float aDeltaTime);
    void Render();

private:
};

class Item : public ObjectBase
{
public:
    Item();

    void Update(float aDeltaTime);
    void Render();

private:
};

class Resource : public ObjectBase
{
public:
    Resource();

    void Update(float aDeltaTime);
    void Render();

private:
    int myAmount{};
};
```

```

struct Slot
{
    int index{};
    ObjectBase* object;
};

enum class InventoryType
{
    None,
    Inventory,
    PlayerInventory,
    Count
};

```

```

class Inventory
{
public:
    Inventory();

    void Init(int aSize, InventoryType aType);
    void Update(float aDeltaTime);
    void Render();

    void AddObject(ObjectBase* aObject);
    void AddObjectAt(int index, ObjectBase* aObject);
    void RemoveObject(ObjectBase* aObject);
    void RemoveObjectAt(int index, ObjectBase* aObject);
    ObjectBase* GetObjectAt(int index);

    void MoveObjectTo(int aIndexA, int aIndexB);
    void SplitResource(int aIndexA, int aIndexB);
    void UndoAction();

    void ToggleInventory();
    bool GetIsOpen() const;

private:
    InventoryType myType;
    int mySlotSize;
    bool myIsInventoryOpen;

    std::vector<Slot> myObjects;
    std::vector<Inventory*> myInventoryStates;
};

```

```

class PlayerInventory
{
public:
    PlayerInventory();

    void Init();
    void Update(float aDeltaTime);
    void Render();

    void MoveIntoOtherInventory(Inventory aInventoryA, ObjectBase* aObjectA,
                                Inventory aInventoryB, ObjectBase* aObjectB);

    Inventory* GetInventory(std::string);
    ObjectBase* GetObjectAt(const Inventory& aInventory, int aIndex);

    void ToggleInventory();
    bool GetIsOpen() const;

private:
    Inventory myInventory;
    Inventory myActiveInventory;
};

```

```

class Player
{
public:
    Player();

private:
    PlayerInventory myPlayerInventory;
};

class GameWorld
{
public:
    GameWorld();
    ~GameWorld();

    void Init();
    void Update(InputManager& inputManager, float aTimeDelta);
    void Render();

private:
    Player myPlayer;
};

```

Vanliga begrepp

Sök reda på innebörden följande förkortningar och termer (inom mjukvaruutveckling och programmering). Beskriv kort deras innebörd och när man kan vilja använda/tänka på dem samt ge ett exempel på när de inte behövs, inte är en bra idé eller kan vara en ”fälla”. För varje punkt ange minst en källa. Tips: leta gärna både efter allmän information och relaterat till C++ och spelprogrammering.

Polymorfism - Tillåter en variabel att ha mer än en typ och möjliggör en operation att bli utförd på värden av mer än en typ. Ett bra användningstillfälle är när man vet att flera klasser kommer se lika ut och utföra liknande operationer, då man kan använda sig av exempelvis inheritance för att kunna kalla på en funktion i en klass som är av olika typer men har samma basklass. Detta gör att man bland annat undviker duplicerad kod. Man kan även använda det för att göra interfaces för klasser som då måste implementera en eller flera av basklassens funktioner (pure virtual i c++). Ett exempel när de inte behövs är när det inte är många klasser som delar samma typ av data. En fälla kan vara att man overengineerar ett program där polymorfism bara bidrar till komplexiteten av programmet utan någon riktig fördel.

https://www.reddit.com/r/learnprogramming/comments/195p233/i_dont_understand_polymorphism_can_someone_please/

[https://en.wikipedia.org/wiki/Polymorphism_\(computer_science\)](https://en.wikipedia.org/wiki/Polymorphism_(computer_science))

<https://stackoverflow.com/questions/8194250/polymorphism-in-c>

PIMPL - eller Pointer to IMPLementation är en teknik för att gömma implementationer där en publik klass wrapper en struct eller klass som inte syns från utsidan av ett bibliotek som publik klassen är en del av. Detta kan användas för att minska kompileringstider och dölja privata implementations detaljer från användare av ett bibliotek. Exempel på när det inte behövs är när man har små enkla klasser då det lägger till komplexitet utan någon stor fördel.

https://www.reddit.com/r/cpp/comments/r4atq2/when_to_use_the_pimpl_idiom/

<https://stackoverflow.com/questions/60570/why-should-the-pimpl-idiom-be-used>

POD - Plain old data är en datastruktur som är representerad som passiva samlingar av instansvariabler utan att använda objekt orienterade funktioner. Detta betyder alltså att POD är en class eller struct utan constructors, destructors eller virtuella funktioner. Man vill använda POD eftersom att det är enkelt för kompilatorn att optimera och kan säkert skickas mellan c och c++ kod. POD kan vara en fälla om resurshantering behövs om till exempel strukturen ska äga dynamiskt minne, filer eller andra resurser.

<https://stackoverflow.com/questions/146452/what-are-pod-types-in-c>

https://en.wikipedia.org/wiki/Passive_data_structure

RAII - Resource acquisition is initialization or RAII, är en teknik som binder livscykeln av ett objekt. En resurs är något som måste hämtas/allokeras innan den kan användas, finns i begränsad mängd och måste frigöras när den inte längre behövs (Allokerat heap-minne, öppna filer, låsta mutex:ar nätverksanslutningar mm). Fördelarna med RAII är automatisk resurshantering, exception säkerhet, inga minnesläckor och enklare kod. Många c++ klasser använder RAII såsom std::string som hanterar minne automatiskt, std::vector hanterar dynamiskt minne, std::unique_ptr samt std::shared_ptr hanterar dynamiskt allokerade objekt/data. RAII kan vara en fälla när manuell kontroll behövs om man vill frigöra en resurs tidigare än objektets scope, samt att det blir performance overhead eftersom varje objekt kommer köra constructor/destructor.

<https://stackoverflow.com/questions/2321511/what-is-meant-by-resource-acquisition-is-initialization-raii>

[https://en.wikipedia.org/wiki/Resource_acquisition_is_initialization#:~:text=Resource%20acquisition%20is%20initialization%20\(RAII,is%20tied%20to%20object%20lifetime.](https://en.wikipedia.org/wiki/Resource_acquisition_is_initialization#:~:text=Resource%20acquisition%20is%20initialization%20(RAII,is%20tied%20to%20object%20lifetime.)

<https://en.cppreference.com/w/cpp/language/raii.html>

Rule of zero/three/five - Rule of three är tumregler i c++ för resurshantering och exception-säkerhet. Om en klass har definierat en av dessa så bör de definiera alla tre: Destructör, copy constructor, copy assignment operator. Anledningen är att om en inte fungerar på grund av kompilators automatiska implementation så fungerar den förmodligen inte för de andra heller. En fälla kan vara om klassen inte behöver hantera resurser, då kan man följa rule of zero.

https://en.cppreference.com/w/cpp/language/rule_of_three.html

https://www.reddit.com/r/cpp/comments/ys6gkk/the_rules_of_three_five_and_zero/

[https://en.wikipedia.org/wiki/Rule_of_three_\(C%2B%2B_programming\)](https://en.wikipedia.org/wiki/Rule_of_three_(C%2B%2B_programming))

Class invariant - En class invariant är ett villkor eller en uppsättning villkor som alltid måste vara sanna för ett objekt under hela dess livstid. Man vill använda class invariants för att förstå vilka objekt som är "giltiga" samt hitta buggar genom assertions. Class invariants kan vara en fälla om invarianten är för begränsande vilket gör att kod inte kan köras.

https://en.wikipedia.org/wiki/Class_invariant#:~:text=The%20class%20invariant%20constraints%20the,before%20a%20public%20function%20ends.

<https://www.geeksforgeeks.org/cpp/what-is-class-invariant/>

SoA (Structure of arrays) - En minneslayout där elementen i en struct separeras i parallella arrays, en per fält. Istället för att lagra hela objekt tillsammans, grupperas alla värden separat, såsom alla X-värden, alla Y-värden osv. Man vill använda SoA för att optimera sin kod, som SIMD-optimering då moderna CPUs kan processa 4-16 float samtidigt. Om man bara

behöver läsa en av värdena, exempelvis alla X-värden, så läses mindre data. Det kan vara en fälla att använda SoA när man vill jobba objektorienterat eftersom inkapsling bryts och det kan göra det svårare att arbeta med.

<https://stackoverflow.com/questions/72840933/soa-structure-of-array-vs-aos-array-of-structure>

https://en.wikipedia.org/wiki/AoS_and_SoA

Data Locality - En optimeringsteknik som går ut på att organisera data i minnet så att det ligger sammanhängande (contiguous) i den ordning det bearbetas, vilket utnyttjar CPU-cachens förmåga att ladda närliggande data snabbt. När CPU:n läser en byte från RAM laddar den automatiskt en hel cache line (64-128 bytes av närliggande data). Målet är att organisera data så att allt du behöver finns i den cache line:n. Genom att exempelvis använda sig av ECS (Entity Component System) och data locality kan man processa hundratals entities varje frame snabbt. Data locality-tekniken används även ofta för kollisionsdetektion, rendering eller fysik, alltså för loopar som är kritiska för performance samt iterationer över stora mängder homogen data.. Data Locality kan vara en fälla för all kod som körs sällan samt att det ibland kan förstöra nödvändiga abstraktioner för minimal vinst.

<https://gameprogrammingpatterns.com/data-locality.html>

Immutability - Immutability innebär att ett objekts tillstånd inte kan ändras efter att det har skapats. Ett immutable objekt är alltså "oföränderligt" när det väl existerar kan dess data aldrig modifieras. Det kan användas för thread-safe data, configuration, event logs och undo/redo-system. En fälla är performance eftersom det skapar nya objekt vid varje "ändring" istället för in-place mutation, vilket är dyrt för stora datastrukturer som uppdateras ofta (t.ex. game state varje frame).

[https://en.wikipedia.org/wiki/Immutable_object#:~:text=In%20object%2Doriented%20\(OO\),modified%20after%20it%20is%20created.](https://en.wikipedia.org/wiki/Immutable_object#:~:text=In%20object%2Doriented%20(OO),modified%20after%20it%20is%20created.)

https://www.reddit.com/r/ProgrammingLanguages/comments/1cumxdd/at_a_low_level_what_is_immutability_really/

Higher-order function - En higher-order function är en funktion som tar andra funktioner som argument eller returnerar en funktion. Detta gör att man kan använda abstraktion och används för callbacks, event handlers, sorting med custom comparators och funktionell programmering (map, filter, reduce). Ett exempel när det inte behövs är i tight performance-kritiska loopar där function pointer overhead är för dyr. En fälla är att det kan bli svårare att debugga med djupa call stacks och att virtual function calls eller lambdas kan orsaka cache misses.

https://en.wikipedia.org/wiki/Higher-order_function#:~:text=In%20mathematics%20and%20computer%20science,a%20function%20as%20its%20result.

https://www.reddit.com/r/learnprogramming/comments/1itdc9b/why_to_use_higher_order_functions/

DRY - DRY är principen att undvika duplicerad kod genom att extrahera gemensam logik till återanvändbara funktioner eller klasser. Detta används när samma logik upprepas på flera ställen, vilket minskar underhåll och buggrisk. Ett exempel när det inte behövs är när kod som ser likadan ut faktiskt representerar olika koncept som råkar ha liknande implementation. En fälla är over-abstraction där man forcerar DRY och skapar onödig koppling mellan orelaterade delar, vilket gör kod svårare att förstå och ändra.

https://en.wikipedia.org/wiki/Don%27t_repeat_yourself

https://www.reddit.com/r/programming/comments/xn14lx/why_dry_is_the_most_overrated_programming/

KISS - KISS är principen att enkelhet är bättre än komplexitet där lösningar ska vara så enkla som möjligt men inte enklare. Detta bör användas alltid för att göra kod lättare att förstå, underhålla och debugga. Ett exempel när det inte passar är när problemet genuint är komplext och en "enkel" lösning bara skjuter komplexiteten till användaren. En fälla är att överdriven simplicity kan leda till naiv kod som inte skalar eller hanterar edge cases eftersom ibland är viss komplexitet nödvändig.

https://en.wikipedia.org/wiki/KISS_principle

https://www.reddit.com/r/dotnet/comments/1384rhy/kiss_principle/

YAGNI - YAGNI är principen att inte implementera features förrän de faktiskt behövs, vilket innebär att man ska undvika spekulativ generalisering. Detta används för att undvika att bygga komplex infrastructure för hypotetiska framtida behov som kanske aldrig kommer. Ett exempel när det inte passar är när man designar game engines eller API:er där framtida extensibility måste planeras in från början. En fälla är att alltid följa YAGNI kan leda till arkitektur som är omöjlig att utöka senare, vilket kräver stora refactorings.

https://en.wikipedia.org/wiki/You_aren%27t_gonna_need_it

https://www.reddit.com/r/ExperiencedDevs/comments/11vonwg/yagni_is_a_good_principle_but_many_devs_miss_the/

SOLID (skriv om varje bokstav för sig):

S - (Single responsibility principle) En klass ska ha ett enda ansvar. Detta används när klasser blir för stora med många orelaterade metoder. Ett exempel när det inte behövs är för små utility-klasser. En fälla är över-fragmentering med för många tiny klasser.

O - (Open/Closed Principle) Klasser ska vara öppna för utökning men stängda för modifiering. Detta används för plugin-system. Ett exempel när det inte passar är enkla klasser

där abstraction är överkill. En fälla är överdriven användning av inheritance som strider mot data locality.

L - (Liskov Substitution Principle) Subklasser ska kunna ersätta sina basklasser utan att förstöra programmets korrekthet. Detta används vid polymorfism. Ett exempel när det inte behövs är när inheritance bara används för code reuse. En fälla är onödigt komplexa hierarkier.

I - (Interface Segregation Principle) Många små interfaces är bättre än ett stort generellt interface. Detta används när olika klienter behöver olika delar av ett interface. Ett exempel när det inte behövs är när alla implementationer behöver alla metoder. En fälla är interface explosion med för många små interfaces.

D - (Dependency Inversion Principle) High-level moduler ska inte bero på low-level moduler utan båda ska bero på abstraktioner. Detta används för testbarhet och decoupling. Ett exempel när det inte behövs är i simple kod utan behov av testing. En fälla är extra indirection som skapar cache misses i performance-kritisk kod.

<https://en.wikipedia.org/wiki/SOLID>

https://www.reddit.com/r/learnprogramming/comments/cr3m01/solid_design_principles_for_everyone/

<https://www.digitalocean.com/community/conceptual-articles/s-o-l-i-d-the-first-five-principle-s-of-object-oriented-design>